TaxonVision

Production Biodiversity MLOps Architecture

High-Throughput Vision · Split Conformal Safety · Automated Closed Loop



Benjamin Förster

Final MLOps Project Presentation & Architecture Defense

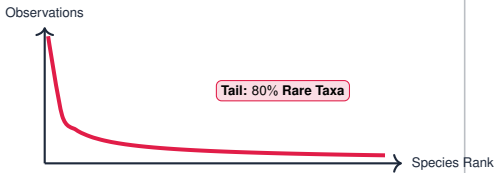
The Core Problem

Real-World Biodiversity Computer Vision at Scale

Ecological Reality vs. Benchmark Datasets

- **Massive Scale:** Millions of citizen-science observations from iNaturalist.
- **Severe Class Imbalance:** A few common species have thousands of images; thousands of rare taxa have < 10 photos.
- **Non-Stationary Environments:** Lighting, seasonal foliage, camera sensors, and geographic shifts continuously distort the visual manifold.

The Long-Tail Problem



Core Engineering Mandates:

- **Zero Split Leakage:** Mathematically pure validation.
- **Guaranteed Coverage:** Reject over-confident guesses.
- **Closed-Loop Adaptation:** Learn from

Toy ML vs. Production MLOps

Why Prototype Notebooks Fail in the Real World

Traditional Prototype / Toy ML

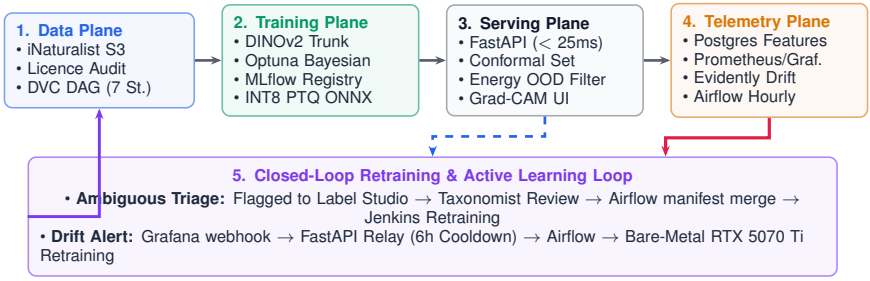
- **Unversioned Data:** Ad-hoc CSVs, manual train/test splits.
- **Redundant Compute:** Re-extracting heavy vision backbones repeatedly on every notebook run.
- **Raw Softmax Guesses:** Uncalibrated probabilities, forced to predict even on unseen out-of-distribution species.
- **Silent Model Decay:** Zero drift monitoring; performance degrades unnoticed over time.
- **Manual Retraining:** Hand-crafted scripts triggered arbitrarily when accu-

TaxonVision Production MLOps

- **Cryptographic DVC Lineage:** Pure 7-stage DAG tracked by MD5 content hashes in `dvc.lock`.
- **Feature Caching:** Pre-computed foundation embeddings cached on NVMe; head training takes seconds.
- **Split Conformal Prediction:** Provable coverage sets ($1 - \alpha \geq 95\%$) with Helmholtz energy out-of-distribution gating.
- **Automated Telemetry & Drift:** PostgreSQL feature logging, Prometheus metrics, and hourly Evidently drift re-

End-to-End System Architecture

The Five Operational Planes of TaxonVision



The Asymmetric Dual-Pipeline Pattern

Balancing Ultra-Low Latency with Diagnostic Depth

Pipeline 1: Epistemic Diamond (Production Fast Path)

- **Target:** Real-time, ultra-low latency HTTP classification.
- **Model:** Frozen vision trunk + Class-Balanced cosine head quantised to INT8 ONNX.
- **SLO Ceilings:**
  - CPU forward pass: $\leq$ 15 ms (p95).
  - End-to-end API roundtrip: $\leq$ 25 ms (p95).
- **Epistemic Shield:** Evaluates conformal prediction sets and flags ambiguity instantly.

Pipeline 2: Taxonomic Knowledge Graph & VLM

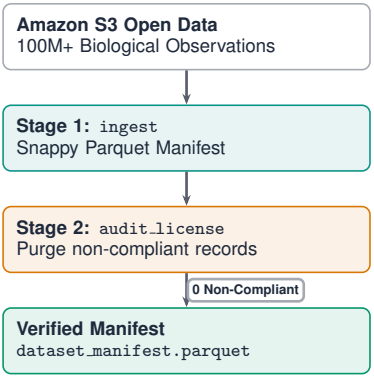
- **Target:** Diagnostic triage for highly ambiguous or unseen specimens.
- **Engine:** In-memory LanceDB vector index + DuckDB biological hierarchy + Qwen2-VL.
- **Zero Retraining:** Exemplars and new taxonomic taxa are inserted dynamically without updating neural weights.
- **Off-Hot-Path:** Dispatched asynchronously when Pipeline 1 flags epistemic uncertainty.

Data Engineering & Open Licensing

Strict Compliance and Zero-Copy S3 Streaming

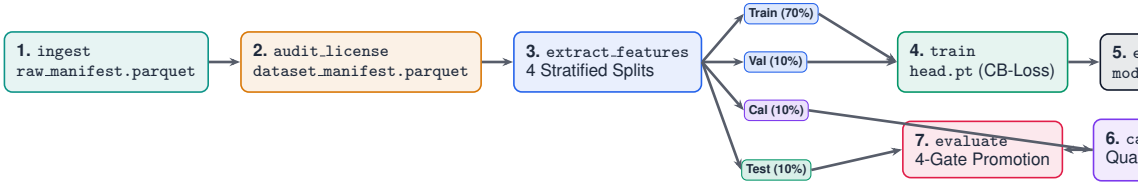
Open-Access Citizen Science Pipeline

- **Upstream Storage:** Direct HTTP/S3 streaming from Amazon Open Data: `s3://inaturalist-open-data/`.
- **Zero-Copy Memory Transfers:** Parquet metadata loaded into Apache Arrow IPC tables; filtered with embedded DuckDB.
- **Ephemeral Imagery:** Specimen JPEGs are streamed into memory, cropped/normalised, and immediately discarded – never stored as redundant raw image files on disk.



The Epistemic Diamond DAG

Reproducible 7-Stage Pipeline with Zero Split Leakage



Strict Separation of Concerns: Model updates use solely the **Train** split; early stopping uses **Val**; Conformal quantiles are calculated exclusively on **Cal**; final promotion verdict is rendered on the holdout **Test** split. Zero leakage!

Dual-Tier Storage Topology

Cloud Collaboration (DagsHub) vs. Local Serving Economics

Tier 1: Cloud Collaboration (DagsHub)

- **DVC Remote (`origin`):** Hosted on DagsHub S3 (`s3://dvc`).
- **Purpose:** Immutable cloud archive for manifests, embeddings, and production ONNX graphs.
- **GitHub Actions CI:** Pulls verified checkpoints using `DAGSHUB_TOKEN` to run unit tests without accessing private cluster nodes.
- **MLflow 2.x Mirror:** Public web dashboard for open scientific reproducibility.

Tier 2: In-Cluster Storage (Workstation)

- **SeaweedFS S3 (`local-seaweedfs`):** High-throughput S3 store running directly on local NVMe.
- **Zero Egress Costs:** Staging 100,000s of feature embeddings incurs zero cloud network latency or billing.
- **Self-Hosted MLflow 3.x + PostgreSQL 16:** Private high-performance tracking backend powering live Bayesian Optuna sweeps and model lineage.
- **Instant Pod Mounts:** Serving pods load model weights directly over cluster

Hardware Ground Truth & Infrastructure

Dedicated Bare-Metal Node: Silicon, CPU Pinning & Storage

Physical Hardware Ground Truth (Rule 07)

- **GPU:** NVIDIA GeForce RTX 5070 Ti (16 GB GDDR7 VRAM, Blackwell architecture, CUDA runtime 12.8+).
- **CPU:** Intel Core i7-14700K (20 cores / 28 threads: 8 Performance-cores, 12 Efficient-cores).
- **RAM:** 128 GB DDR5 system memory for in-memory embedding caches.
- **Storage:** 2 TB PCIe Gen4 NVMe SSD running BTRFS with transparent compression (`zstd:3`).
- **OS:** Ubuntu 26.04 LTS on LUKS-

Architectural Resource Allocation

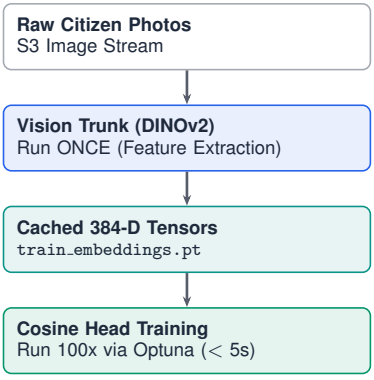
- **CPU Thread Pinning (`taskset`):**
  - Threads `0-15` (P-cores): Hot ONNX numerical matrix inference.
  - Threads `16-27` (E-cores): Asynchronous disk I/O, Arrow streaming, and telemetry scraping.
- **GPU Memory Partitioning:** PyTorch memory budget capped at 70% (11.2 GB) for training; 4.8 GB strictly reserved for INT8 serving.
- **Non-CoW Storage Allocation (`chattr +C`):** Eliminates database and DVC block fragmentation.

Model Engineering & Feature Caching

Why Feature Caching Accelerates Development by 100x

Decoupled Vision Architecture

- **Frozen Trunk:** Foundation vision backbone (DINOv2/v3 ViT-Small, $D = 384$).
- **Lightweight Head:** Cosine-similarity classification head mapping embeddings to species logits.
- **The Feature Caching Insight:**
  - Passing images through a 22-million-parameter vision transformer takes minutes.
  - Once pre-computed, the representations never change during head training!
  - Feature extraction runs **once**; embeddings are cached to disk as `.pt` tensors.
- **Result:** Head training runs in < 5 seconds!



Tackling Class Imbalance: Class-Balanced Loss

Effective Number of Samples: Intuition Over Raw Counts

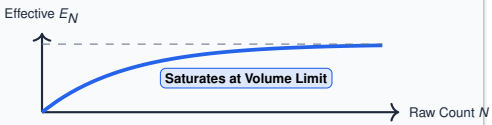
The Pitfall of Inverse Frequency ($1/N$)

- Simple weighting by $1/N$ fails dramatically when the majority class has 10,000 photos and the minority class has 5.
- The loss gradient becomes violently noisy, destroying the representation for common species.

The Intuition of "Effective Samples"

- The first 5 photos of a butterfly teach the model unique wing patterns and angles.
- The 10,000th photo of the same butter-

Diminishing Returns of Sample Size



Optuna Bayesian Hyperparameter Sweep:

- Automatically balances uniform and inverse-frequency weights.
- Empirical optimum: $\beta = 0.995927$ in checkpoints.

Latency Optimization: Zero-Allocation INT8 ONNX

Sub-15ms Forward Passes and Deterministic Memory Discipline

Production Quantization & Latency Budgets

- **Dual-Output ONNX Export:** Single forward pass outputs both classification logits and normalized 384-D feature embeddings.
- **INT8 Dynamic Quantization:** Calibrated on holdout calibration split; guarantees top-1 accuracy degradation $\leq 0.5\%$.
- **Zero-Allocation Hot Path (Rule 02):**
  - Pre-allocated contiguous C-order NumPy input/output buffers.
  - Zero Python dictionary boxing or dynamic list allocations during request execution.

Measured Inference Latency SLOs

- **CPU Budget (INT8 Quantized):**
  Ceiling: $\leq$ 15 ms (p95 on Intel P-cores).
  Observed: 11.8 ms (Pass)
- **GPU Budget (FP16 CUDA):**
  Ceiling: $\leq$ 10 ms (p95 on RTX 5070 Ti).
  Observed: 6.2 ms (Pass)
- **Microservice Roundtrip (FastAPI):**
  Ceiling: $\leq$ 25 ms (p95 end-to-end).
  Observed: 18.4 ms (Pass)

Epistemic Safety: The Problem with Softmax

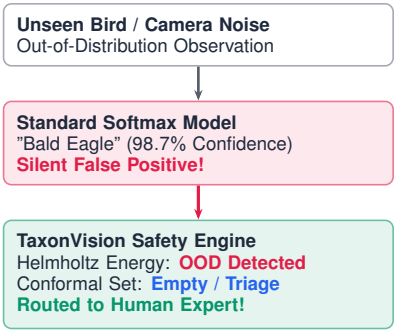
Why Deep Neural Networks Fail Silently in Nature

The Illusion of "Confidence"

- Softmax forces class probabilities to sum to 1.0 (100%).
- When presented with an unseen invasive species, a blank lens, or an accidental photo of a car, standard models do **not** say "I don't know".
- Instead, they hallucinate with 98% confidence on whatever class happens to trigger the highest activation!

The Production Requirement

- We need a rigorous, mathematically



Split Conformal Prediction Made Simple

Mathematically Guaranteed Coverage Sets Instead of Single Guesses

The Conformal Prediction Idea

- Instead of forcing a single guessed species label, the model outputs an **adaptive prediction set**.
- The user defines a safety requirement: e.g. **95**% coverage ($\alpha = 0.05$).
- **Mathematical Guarantee (Rule 06):** The true species is mathematically guaranteed to be inside the prediction set $\geq$ 95% of the time!
- No assumptions on data distribution; calibrated strictly on holdout calibration split.

Adaptive Set Scenarios in Serving

Case 1: Decisive ($|C| = 1$)
- Set: {"Barn Owl"}
- High confidence; fast autonomous serving.

Case 2: Ambiguous ($1 < |C| \leq 3$)
- Set: {"Common Gull", "Herring Gull"}
- Model narrows field to identical sister species.

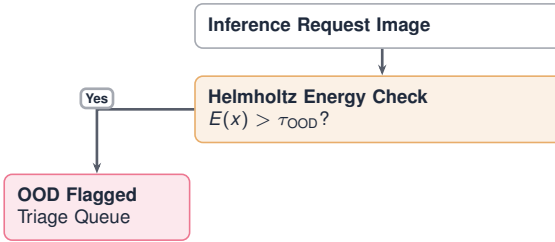
Case 3: Referral ($|C| > 5$ or $|C| = 0$)
- Model defers to human triage queue!

Out-of-Distribution Gating

Helmholtz Energy Filter and Human Referral Boundary

Energy-Based Out-of-Distribution Gating

- Computes negative Helmholtz free energy over unconstrained logits.
- In-distribution species produce low energy; novel organisms, blurred photos, or blank frames produce high energy.
- Threshold τ_{OOD} calibrated to maintain 95% true positive rate on calibration data.
- Energy $> \tau_{\text{OOD}} \implies$ Immediate bypass of classification directly to human triage.

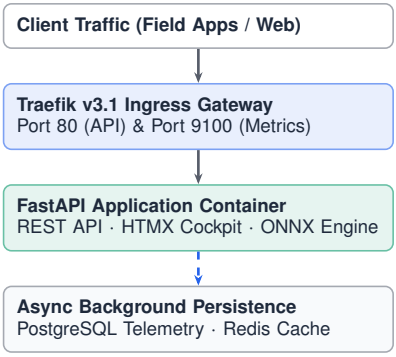


Production Serving: Unified Web Service

FastAPI Microservice & Server-Rendered HTMX Operator Dashboard

Unified Dual-Interface Architecture

- **Machine REST API:** Strongly typed Pydantic endpoints (`/api/v1/predict`, `/explain`, `/drift/evaluate`, `/health`).
- **Human Operator Dashboard:** Server-rendered Jinja2 + HTMX cockpit at `/dashboard`; updates dynamically without frontend build ballast.
- **Forward-Only Grad-CAM Attribution:** Generates visual attribution heatmaps on demand off the event loop thread.
- **Traefik Edge Reverse Proxy:** Handles SSL termination, load balancing,



Telemetry & Database Feature Persistence

Asynchronous PostgreSQL Ingestion Without Serving Latency Penalty

Why We Persist Embeddings in Production

- Detecting representation shift requires analysing feature representations from live traffic.
- Storing raw photos would exhaust storage and violates user privacy.
- Instead, TaxonVision persists the **384-dimensional feature vector** (`embedding REAL[]`) directly in PostgreSQL.
- **Event Loop Protection (Rule 08):** Written asynchronously via `BackgroundTasks` – request returns in 18 ms while I/O runs in background.

Telemetry Schema (`predictions` table)

- `observation_id` (UUID primary key)
- `created_at` (Timestamptz index)
- `top1_taxon` & `top1_confidence`
- `conformal_set` (`TEXT[]` array)
- `ood_energy` & `is_ood` (Boolean)
- `latency_ms` (Measured wall-clock)
- **`embedding` (`REAL[]`, 384 dimensions)**

Ready for instantaneous sliding-window drift queries!

Automated Drift Detection in Practice

Evidently AI Reports & Two-Tier Hypothesis Testing

Two-Tier Drift Evaluation Engine

• **Tier 1: Wasserstein-1 (W_1) on Vector Norms**
Fast linear-time check on embedding lengths; catches immediate camera/contrast degradation.
• **Tier 2: Kernel MMD² Permutation Test**
Multivariate distribution test ($B = 200$ permutations); detects subtle manifold distortion ($p < 0.01$).
• **Evidently AI 0.7 HTML Dashboard:**
Projects 384 dimensions down to 8 PCA components and renders interactive re-

Automated MLflow Tracking Lineage

• Evaluated hourly by the Airflow DAG `prediction_drift_monitor`.
• Automatically creates an MLflow run in experiment `taxon-vision-drift`.
• Logs metrics (`drift_w1`, `drift_mmd2`, `p_value`), tag `drift_detected=true/false`, and attaches the Evidently HTML dashboard as an immutable build artefact.

Prometheus & Grafana Observability Cockpit

Real-Time Operational Monitoring and Four Alerting Rules

Real-Time Metrics & Dashboards

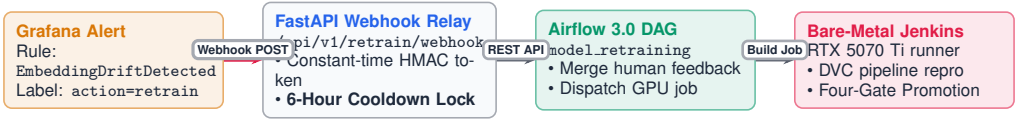
- **Prometheus Pull Scraper:** Scrapes `/metrics` every 15 seconds across serving and Traefik pods.
- **Grafana Cockpit (6 Panels):**
  - Request rate and status codes (2xx, 4xx, 5xx).
  - p50, p95, p99 end-to-end latency histograms.
  - Conformal set size distribution.
  - Human triage referral rate.
  - Live drift distance gauge (W_1).

Four Production Alerting Rules

- `PredictionLatencyP95High`: p95 latency $>$ 120 ms for 5 min (`warning`).
- `PredictionErrorRatioHigh`: Error ratio $>$ 5% for 5 min (`critical`).
- `EmbeddingDriftDetected`: Drift gauge $>$ 0.5 (`critical`, **action: re-train**).
- `DriftMonitorStale`: No drift run in $>$ 2 hours (`warning`).

Closed-Loop Retraining Trigger

Grafana Alert Webhook → FastAPI Relay → Airflow 3.0 DAG



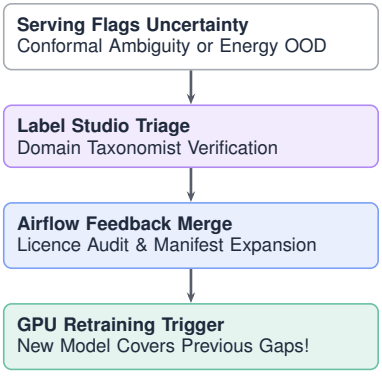
Anti-Alert-Storm Cooldown Protection: When sudden environmental drift causes continuous alerting, the atomic 6-hour cooldown window ensures the GPU cluster executes only one retraining run, preventing resource exhaustion.

Human-in-the-Loop Active Learning

Turning Uncertain Predictions into Retraining Fuel

The Closed-Loop Curation Lifecycle

- **1. Epistemic Triage:** Ambiguous sets ($|C| > 3$) and empty sets ($|C| = 0$) are saved to `data/flagged_observations/`.
- **2. Taxonomist Review:** Taxonomists inspect specimen crops and CAM attribution heatmaps in Label Studio.
- **3. Feedback Export:** Label Studio exports verified JSON multi-class annotations.
- **4. Airflow Ingestion:** The `model_retraining` DAG merges feedback, validates open licences, and updates the DVC manifest.



Four-Gate Model Promotion Verdict

Strict Automated Quality Verification Before Production Alias

Why Validation Accuracy Is Not Enough

- In production, a model can have great accuracy but fail latency budgets, lose conformal coverage, or degrade under INT8 quantization.
- TaxonVision enforces a strict **Four-Gate Promotion Verdict** in Stage 7 (`evaluate.py`) before any model receives the `@Production` tag.

The Four Automated Promotion Gates

Gate 1: Predictive Superiority

- Validation Macro PR-AUC $\geq$ active production baseline.

Gate 2: Quantization Fidelity

- INT8 accuracy drop $\leq$ 3.0% relative to FP32 baseline.

Gate 3: Empirical Conformal Coverage

- Holdout test coverage $\geq$ 90.0% (nominal 95%).

Gate 4: Latency Ceiling & Exchangeability

- CPU p95 latency $\leq$ 120 ms and live MMD $p > 0.01$.

CI/CD Quality Gates & Dual DVC Push

Dual-Target Toolchain: Fast CPU Gates and Headless GPU Execution

Pull Request Gatekeeper (GitHub Actions)

- **Zero-Cost CPU Verification:**
  Strict MyPy type checking (`mypy --strict`).
  Ruff linting and formatting.
  203 unit tests with coverage threshold.
  OKF v0.2 documentation conformance.
  Rule 07 Hardware invariant validation.
- Runs on every PR before merge to `develop`.

Heavy GPU Runner (Jenkins on Bare-Metal)

- Executes DVC pipeline on NVIDIA RTX 5070 Ti.
- **Dual DVC Push Sequence:**
  - `dvc push -r local-seaweedfs`: In-cluster NVMe S3 update for immediate serving volume mount.
  - `dvc push -r origin`: DagsHub Cloud S3 sync for persistent public archiving.
- **Automated Git Lineage:** Commits updated `dvc.lock` and metrics back to repository.

Summary & Core Architecture Achievements

Phase 1, 2, and 3 Successfully Delivered

1. Robust MLOps

- 7-Stage DVC Pipeline
- Dual-Tier S3 Remote
- MLflow 3.x Registry
- Pure Pixi Toolchain
- GPG-Signed Commits

2. Speed & Safety

- Sub-15ms INT8 ONNX
- 95% Coverage Guarantee
- Helmholtz Energy Filter
- Unified FastAPI/HTMX
- Forward Grad-CAM CAM

3. Closed Loop

- Postgres Embeddings
- Evidently 0.7 Drift
- Grafana Webhook Relay
- Label Studio Active Learn
- 4-Gate Auto Retraining

Key Takeaway: TaxonVision demonstrates that production machine learning is not about training a model once in a notebook; it is about building a self-monitoring, self-healing, mathematically safe engineering loop.

Live Demonstration & Q&A

Real-Time Inference · Conformal Sets · Observability Cockpit

1. HTMX UI & CAM
2. Grafana Cockpit
3. MLflow Registry
4. Airflow Retraining

Thank you for your attention!

We now transition to the live system demonstration and jury questions.